



SAPIENZA
UNIVERSITÀ DI ROMA

Emulatore CUDA

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Laurea Triennale in Ingegneria Informatica

Gabriele Guido

Matricola 2144286

Relatore

Prof. Giorgio Grisetti

Anno Accademico 2023/2024

Emulatore CUDA

Tesi di Laurea. Sapienza Università di Roma

© 2026 Gabriele Guido. Tutti i diritti riservati

Questa tesi è stata composta con $\text{\LaTeX}$ e la classe Sapthesis.

Email dell'autore: guido.2144286@studenti.uniroma1.it

Sommario

La programmazione in CUDA (*Compute Unified Device Architecture*) è il modo più efficace per scrivere programmi eseguibili su GPU (*Graphics Processing Unit*). L'architettura delle schede grafiche consente numerose operazioni parallele, riducendo i tempi di esecuzione e aumentando il **throughput** (attività completate nell'unità di tempo). Negli ultimi anni l'utilizzo di questa tecnologia si è diffuso ulteriormente grazie alla velocità di esecuzione di calcoli sulle matrici e programmare in questa modalità è diventata quindi una competenza richiesta. Il suo punto di forza tuttavia è ciò che rende la progettazione degli algoritmi meno semplice: i dati sono elaborati in parallelo e senza ordine perciò durante l'esecuzione potrebbero verificarsi delle dipendenze dati che, se non trattate correttamente, porterebbero a risultati sbagliati. In CUDA abbandoniamo il concetto di esecuzione sequenziale: in ogni istante un numero finito di thread esegue lo stesso codice su una porzione dell'input dato al programma. Senza un ordine tra questi diventa necessario disegnare algoritmi in cui i thread lavorino in modo indipendente tra loro.

Il progetto nasce dal bisogno di programmare in CUDA anche senza una scheda grafica sul proprio PC e consiste in una libreria in cui ci sono le funzionalità essenziali per testare la correttezza di un algoritmo destinato al calcolo parallelo su GPU. A livello prestazionale non c'è alcun vantaggio, la vera utilità sta nella scrittura e verifica del funzionamento di algoritmi destinati al calcolo parallelo.

Indice

1	CUDA	1
1.1	In poche parole	1
1.2	Griglia, Blocchi e Thread	1
2	Algoritmi	3
2.1	ReductionSum	3
2.2	PrefixSum	3
3	MyCuda	5
3.1	Parametri	5
3.2	Strumenti Principali	5
3.2.1	MyCudaItem	5
3.2.2	MyCudaKernelLaunch	6
3.2.3	MyCudaMemSwap	7
3.3	Algoritmi	7
3.3.1	MyCudaReduction	8
3.3.2	MyCudaPrefixSum	9

CUDA

1.1 In poche parole

Per capire come funziona l'esecuzione di un programma su una GPU dobbiamo abbandonare la concezione sequenziale e iniziare a pensare a cosa succede nello spazio più che nel tempo. Prenderemo come esempio di input da processare dei buffer di N numeri (interi). Supponiamo di voler eseguire una moltiplicazione per uno scalare S in ogni cella di $dest[N]$ e salvare il risultato su $src[N]$. Su una CPU non serve altro che iterare l'operazione su tutte le N posizioni del nostro vettore.

```
1   for(int i = 0; i < N; i++)  
2       dest[i] += S * src[i];
```

Volendo ottenere lo stesso risultato su una GPU, iterare in questo modo non avrebbe senso: per sfruttare la potenza di calcolo dobbiamo ottimizzare l'esecuzione. In questo esempio dobbiamo pensare che le operazioni di moltiplicazione stavolta vengono fatte su tutte le celle contemporaneamente: ogni thread esegue lo stesso codice su una singola posizione dell'array.

```
1       dest[idx] += S * src[idx];
```

In questo modo il costo di questa semplice operazione passa da essere lineare ($O(n)$) a costante ($O(1)$).

1.2 Griglia, Blocchi e Thread

Per poter gestire input con dimensione maggiore dei core (e quindi thread) che abbiamo a disposizione, l'esecuzione viene spezzata in blocchi: ogni blocco è grande quanto il numero massimo di thread a disposizione. L'input viene processato come una griglia di blocchi quindi CUDA mette a disposizione dei programmatori le informazioni come numero di thread (in un blocco), numero di blocco (nella griglia) e dimensione del blocco. In questo modo possiamo tenere conto dello spostamento

spaziale della nostra esecuzione, processando il nostro input come se fosse composto da tante "mattonelle" (da non confondere con le celle di un array).

Algoritmi

Nella libreria sono presenti già due algoritmi usati per testare il funzionamento delle funzioni. Vediamo in generale cosa sono e a che servono. L'implementazione nella libreria si trova al capitolo di MyCuda, qui mostrerò una equivalente (sempre in C) sequenziale per capirne il meccanismo.

2.1 ReductionSum

Prende in input un vettore (numerico) e restituisce la somma di tutte le componenti.

```
1  int SequentialReductionSum(int* src, int size){  
2      int sum = 0;  
3      for(int i = 0; i<size,i++){  
4          sum += src[i];  
5      }  
6      return sum;  
7  }
```

2.2 PrefixSum

Prende in input un vettore (numerico) e scrive in ogni cella *i-esima* la somma dei valori nelle posizioni da 0 a i del vettore.

```
1  void SequentialPrefixSum(int* src, int size){  
2      int sum = 0;  
3      for(int i = 0; i<size; i++){  
4          sum += src[i];  
5          scr[i] = sum;  
6      }  
7  }
```


MyCuda

La libreria MyCuda contiene delle funzionalità utili alla progettazione e simulazione del calcolo parallelo. L'esecuzione effettiva è gestita tramite un numero limitato di thread che simulano i core di una GPU. Il funzionamento generale è gestito tramite una struttura dati *MyCudaItem* al cui interno troviamo i buffer di memoria (con rispettiva dimensione) e il puntatore alla funzione scritta in "*pseudo-CUDA*" che verrà eseguita simulando i core della GPU. Nella libreria la maggior parte delle funzioni sono delle operazioni ausiliare che facilitano la fase di sviluppo e verifica: gestione dei 2 buffer (creazione, stampa ed eliminazione), il calcolo di valori speciali (dimensione griglia) e verifica di condizioni necessarie (dimensione buffer). Il cuore della libreria consiste nella funzione che simula la chiamata al *Kernel* della GPU e tutta la gestione, tramite strutture dati dedicate, di indici, parametri e argomenti usati dai thread. Al di là quindi delle varie funzionalità operative, l'essenza di questa libreria è la possibilità di scrivere funzioni (in C) similmente a come si farebbe usando CUDA.

Nota: L'intera libreria è stata pensata per trattare array di interi ma la meccanica può essere facilmente estesa ad altri tipi di dato.

3.1 Parametri

Nell'header *myCuda.h* troviamo le **macro** usate nella libreria.

- **BLOCK_DIM:** dimensione di ogni blocco e quindi il numero massimo di thread generati;
- **DEBUG:** quando è maggiore di 0 attiva le stampe intermedie nelle funzioni, rendendo più facile l'ispezione;
- **SLEEPING_TIME:** nei test ho rallentato le stampe con delle `sleep()` per rendere più leggibile il terminale;
- **RANDOM_RANGE:** indica fino a che numero possono arrivare i numeri pseudocasuali generati come input per i test.

3.2 Strumenti Principali

3.2.1 MyCudaItem

Ho scelto di usare una struttura dati per salvare gli elementi più ricorrenti e rendere tutte le funzioni consistenti con questo tipo di dato. Le funzioni di libreria avranno


```

9         //kernel launch:
10        pthread_create(&threads[tid], NULL, cuda_item->
            kernel_fn, &thread_arguments[tid]);
11    }
12    for(int tid = 0; tid < BLOCK_DIM; tid++){
13        pthread_join(threads[tid], NULL);
14    }
15    }
16    }

```

Una volta entrati nella funzione, i parametri vengono estratti dalla **struct** e salvati localmente (vedere `MyCudaReductionKernel`).

3.2.3 MyCudaMemSwap

In molti algoritmi, prima di arrivare al risultato, ci sono diversi passaggi intermedi in cui si salvano risultati da usare successivamente. Per evitare di consumare molto spazio usiamo i due array di sorgente e destinazione in modo alternato. Questa funzione non fa altro che invertire, all'interno della struttura `MyCudaItem`, i puntatori alle due zone di memoria. Ad esempio nel caso di `ReductionSum` si parte da `src` con dimensione `n`, si calcolano le somme a coppie e si salvano gli `n/2` risultati in `dest`, al passaggio successivo il nuovo buffer sorgente è quello in cui abbiamo appena salvato le somme, quindi invertiamo i due array e ripetiamo finchè dopo $\log(n)$ passaggi troviamo un solo elemento che rappresenta il risultato.

```

1  void MyCudaMemSwap(MyCudaItem* cuda_item){
2      assert(cuda_item && "CudaItem not valid");
3      int* aux = cuda_item->src_buffer;
4      cuda_item->src_buffer = cuda_item->dest_buffer;
5      cuda_item->dest_buffer = aux;
6      for(int i = 0; i < cuda_item->buffer_size; i++){
7          cuda_item->dest_buffer[i] = 0;
8      }
9  }
10 }

```

3.3 Algoritmi

Entrambi gli algoritmi modificano direttamente i buffer in input, simulando ciò che avviene in GPU.

3.3.1 MyCudaReduction

MyCudaReduction Questa funzione serve da "guscio" per l'esecuzione effettiva: crea i parametri, gestisce le iterazioni e invoca la (finta) chiamata al kernel. Dopo un primo passaggio il buffer di destinazione sarà grande metà dell'input e conterrà le somme a coppie degli elementi in `src`: a questo punto viene invocata `MyCudaMemSwap()` che inverte `dest` e `src` per il passaggio successivo, e così si itera finché la dimensione del risultato è pari a 1, ovvero la somma totale degli elementi iniziali.

```

1 void MyCudaReduction(MyCudaItem* cuda_item){
2     assert(cuda_item && "cuda_item_not_valid");
3
4     cuda_item->size = cuda_item->buffer_size;
5     cuda_item->size = (cuda_item->size+1)/2;
6     cuda_item->kernel_fn = MyCudaReductionKernel;
7
8     while(1){
9         //kernel launch
10        MyCudaKernelLaunch(cuda_item);
11        if(cuda_item->size>1){
12            cuda_item->size = (cuda_item->size+1)/2;
13            MyCudaMemSwap(cuda_item);
14        }else{
15            break;
16        }
17    }
18 }
```

MyCudaReductionKernel L'esecuzione effettiva su GPU (o meglio la sua simulazione). Una volta estratti i parametri e verificato il giusto posizionamento entro i limiti del buffer si sommano due celle contigue di `src` e viene scritto il risultato della corrispondente posizione in `dest`.

Nota: ogni thread ha 2 celle dedicate e nessun altro interagisce con queste, preservando la correttezza del codice.

```

1 void* MyCudaReductionKernel(void* args){
2     ThreadArgs* info = (ThreadArgs*) args;
3     //estrazione parametri
4     int idx = info->threadIdx+info->blockIdx*BLOCK_DIM;
5     int size = info->size;
6     int* src = info->src;
7     int* dest = info->dest;
8
9     if(idx >= size){
```

```

10     pthread_exit(NULL);
11 }
12 dest[idx] = src[2*idx]+src[2*idx+1];
13 pthread_exit(NULL);
14 }

```

3.3.2 MyCudaPrefixSum

MyCudaPrefixSum Anche stavolta ci serve una funzione di gestione che dall'esterno si occupa di parametri, esecuzioni e chiamate al kernel. Questo algoritmo si divide in due passaggi:

- DownPass: scriviamo in **dest** le somme a coppie dell'input e il risultato viene poi sostituito nella rispettiva posizione pari di **src**.
- UpPass: sommiamo ogni valore di **src** nelle due celle rispettive di **dest**, saltando le prime 2 posizioni.

DownPass viene eseguito finchè la dimensione di **dest** è pari a 1. A quel punto invertiamo i due buffer e risaliamo invocando **UpPass** a ogni passaggio, finchè torniamo alla dimensione iniziale dell'input. Nella funzione gestisco le due chiamate nei momenti diversi semplicemente impostando il campo **kernel_fn** di **MyCudaItem**, sarà compito di **MyCudaKernelLaunch()** invocare la funzione puntata da quel campo.

Nota: Per evitare spreco di spazio, assumo negli algoritmi di avere dei buffer con dimensione potenza di due (verifica effettuata con funzione ausiliaria **MyCudaIsPowerTwo()**). In questo modo posso usare dividere il **dest** con dei sottobuffer contigui la cui dimensione si dimezza a ogni passaggio: con questo meccanismo posso evitare di allocare ulteriore memoria per i passaggi intermedi.

```

1 void MyCudaPrefixSum(MyCudaItem* cuda_item){
2     assert(cuda_item && "cuda_item_not_valid");
3     assert(MyCudaIsPowerTwo(cuda_item->buffer_size) && "Size_
      must_be_a_power_of_2");
4
5     cuda_item->size = cuda_item->buffer_size;
6     cuda_item->size = (cuda_item->size+1)/2;
7     cuda_item->kernel_fn = MyCudaPrefixDownPassKernel;
8
9     int* orig_dests = cuda_item->dest_buffer;
10    int* orig_src = cuda_item->src_buffer;
11
12    while(cuda_item->size>0){
13        MyCudaKernelLaunch(cuda_item);
14        if(cuda_item->size == 1){
15            break;
16        }

```

```
17     cuda_item->src_buffer = cuda_item->dest_buffer;
18     cuda_item->dest_buffer += cuda_item->size;
19     cuda_item->size = (cuda_item->size+1)/2;
20 }
21 cuda_item->size = 2;
22
23 cuda_item->dest_buffer = cuda_item->src_buffer;
24
25 cuda_item->kernel_fn = MyCudaPrefixUpPassKernel;
26
27 while(cuda_item->size*2 < cuda_item->buffer_size){
28     cuda_item->src_buffer = cuda_item->dest_buffer;
29     cuda_item->dest_buffer -= cuda_item->size*2;
30     MyCudaKernelLaunch(cuda_item);
31     cuda_item->size *= 2;
32 }
33
34 cuda_item->dest_buffer = orig_src;
35 cuda_item->src_buffer = orig_dets;
36
37 //ultima chiamata per quando siamo tornati su 2 buffer
   diversi
38 MyCudaKernelLaunch(cuda_item);
39
40 }
```